GET ON THE AIR

MORSE PRACTICE PAGE

EXERCISE AND CONTENT DEVELOPMENT GUIDE

VERSION 1.0

Deleted: <object>

Deleted: THE PATH TO MORSE CODE FLUENCY

Deleted: ¶ ... [1]

Deleted: 8

Deleted: 0

LICW™ is a trademark of the Long Island CW Club Inc.

© 2022-2026 Long Island CW Club Inc. All rights reserved.

This work is the sole property of the Long Island CW Club Inc. (LICW). It may be downloaded and printed for the use of Long Island CW Club Inc. instructors and students. It may not be reproduced on paper or digitally for other purposes without the express consent of the Long Island CW Club Inc.

LICW is committed to ensuring digital accessibility for people with disabilities. We strive to continually improve our curriculum, teaching methods, and training devices to enhance the user experience for all. This document has been optimized for screen readers used by the visually impaired.

Deleted: ed

Formatted: Tab stops: Not at 0.64" + 1.27" + 1.91" + 2.54" + 3.18" + 3.82" + 4.45" + 5.09" + 5.73" + 6.36" + 7" + 7.63" + 8.27" + 8.91" + 9.54" + 10.18"

Deleted: ¶
¶
LICW hosts a Hams with Disabilities Session with Chris VE3RWJ on the first Monday of the month at 7 PM Eastern in Zoom C. Members with any disability that affects learning or operating CW are encouraged to attend. ¶
¶
Members with hearing impairments may benefit from our revolutionary haptic device. Members may learn more here. ¶

Deleted: ¶ ... [2]

Formatted: List Paragraph

PURPOSE

This document provides guidance on the structure and creation of exercises and content for the Morse Practice Page (MPP). It covers the GitHub repository layout, configuration files, content file formats, and special coding techniques used to control voice playback and group randomization.

GITHUB STRUCTURE

> *Important: Always work in the Develop branch. Never make changes directly to the MAIN branch.*

The repository contains a folder called SRC, which holds all relevant subfolders. The four folders you will work with most are:

- configs
- presets
- wordfiles
- wordfilesconfigs

General Guidelines

The following guidelines apply across all file types in the repository:

- Read the LICW MPP Users Guide, available on the LICW website under Academic Downloads. It describes every setting in the MPP user interface in detail.
- In all .json files, the last line must not end with a comma. Every other line must end with a comma.
- Treat all values as case-sensitive. While this is not strictly true in every case, it is safer to assume it always is.
- To add a file: navigate to the target folder, click the "Add File" tab in the upper-right corner, choose "Upload files," and drag your file into the window. Once uploaded, enter a description with the date and your name or initials in the Commit Change box, then click the green "Commit Change" button.
- To delete a file: click the filename, then click the three-dot menu in the upper-right corner and select "Delete file."
- You can monitor the deployment of your changes by opening the "Actions" tab at the top of the GitHub page. This is a two-step process — first the change is processed (green check), then a "pages build and

deployment" task runs (another green check). A red circle indicates a problem at one of these steps.

- After deployment completes, refresh the MPP page for your changes to take effect.
- The discussion below will provide details on all the preset settings but the easiest way to develop a new preset is to configure it in the MPP and then use the SAVE tab to generate a new preset.

SECTION 1: EXERCISE CONFIGURATION

Configs Folder

The configs folder contains a file called wordify.json. This file defines how specific characters, abbreviations, and words are pronounced by the VOICE function. For example, the "," character is spoken as "comma," and "GA" is spoken as "good afternoon."

Each character combination may only have one default pronunciation. If a different pronunciation is needed in a specific context — for example, "GA" should be spoken as "Georgia" rather than "good afternoon" — that override must be applied directly in the content file using special coding. This is covered in Section 2.

Presets Folder

The Presets folder contains two subfolders you will use: configs and sets. You can ignore the others.

Presets → Configs

The configs subfolder holds the individual configuration files for each preset shown in the MPP "PRESET" dropdown list. Each file corresponds to one preset. For example, the file ADV1AFIX.json holds the settings for the Advanced 1 prefix-and-suffix exercises.

Most file names begin with the class level followed by an identifier for the exercise. Some exceptions exist (for example, sudoku20.json). The easiest way to locate a particular config file is to look it up through the sets folder, described below.

Each entry in a config file follows this format:

```
{
  "key":     "wpm",
  "value":   "25",
  "comment": null
},
```

The "key" maps to a specific MPP setting. The "value" is a number, a string, or true/false. The "comment" field is descriptive only and is normally set to null.

The table below maps each config key to its corresponding setting in the MPP interface:

KEY	MPP SETTING
wpm	Character Speed (WPM)
FWPM	Effective Speed (FWPM)
xtraWordSpaceDits	Word Space
volume	Speaker icon (volume)
stickySets	Makes "BK" the default for Sticky Sets
ifStickySets	Sticky Sets on or off
syncWpm	Lock icon — synchronizes WPM and FWPM
hideList	Determines if "More Settings" is open or closed
showRaw	Determines if the cards are shown or not
autoCloseLessonAccordian	Auto-closes the "More Settings" panel
customGroup	Sets the default characters in the Custom Group area

KEY	MPP SETTING
showExpertSettings	Opens or closes the Expert Settings panel
voiceEnabled	Turns the "Voice" checkbox on or off
voiceSpelling	Turns the "Spell" checkbox on or off
voiceThinkingTime	Sets the "Delay Before" time
voiceAfterThinkingTime	Sets the "Delay After" time
voiceVolume	Sets the Voice volume (speaker icon in Expert Settings)
voiceLastOnly	Sets "Last Only" checkbox on or off
voiceRecap	Sets "Arm Recap" on or off
keepLines	Sets "Keep Lines" checkbox on or off
syncSize	Lock icon — synchronizes Min and Max group size
overrideSize	Sets "Override Size" checkbox on or off
overrideSizeMin	Sets Min to the minimum group size
overrideSizeMax	Sets Max to the maximum group size
cardSpace	Toggles Cards on and off (shows or hides cards)
miscSettingsAccordionOpen	Opens or closes the More Settings area
speedInterval	Opens and closes the "Speed Intervals" area
intervalTimingsText	Sets the time values in Speed Intervals (5, 10, 15, 20)
intervalWpmText	Sets the WPM speed in Speed Intervals
intervalFwpmText	Sets the FWPM speed in Speed Intervals
voiceBufferMaxLength	Not currently used?
speakFirst	Toggles "Voice First" on or off

KEY	MPP SETTING
speakFirstRepeats	Sets the number of repeats in the "Repeats" tab
speakFirstAdditionalWordspaces	Sets the time between repeats in Repeat Spacing
isShuffledSet	Sets "Shuffle" on or off
shuffleIntraGroup	Sets intra-group shuffle on or off (no MPP tab)

Presets → Sets

The sets subfolder contains one file per class level (e.g., BC1, BC2). Each file defines the list of presets shown in the MPP dropdown for that class, linking each display name to its config file.

Each entry follows this format:

```
{
  "display":  "Default 25/25",
  "filename": "ADV1BUF.json"
},
```

"display" is the label shown in the MPP preset dropdown. "filename" is the corresponding config file from the configs subfolder.

Wordfiles Folder

The wordfiles folder stores all exercise content files, which may be .json or .txt format. The vast majority are .txt files. Unfortunately file naming is not consistent — some names begin with the class level, others begin with the exercise name, and some use a short alphanumeric identifier. The most reliable way to locate a file is to search for it in the wordlists.json file (described in the next section).

.json files are used when the content needs to embed settings such as minimum and maximum group length, or the exercise duration. These are primarily used in BC1 and BC2.

Text files contain no embedded settings and require a preset or manual MPP configuration. They range from very simple to complex, with optional internal coding to control how character groups are spoken or sequenced. This coding is explained in Section 2.

Wordfilesconfigs Folder

This folder contains the critical wordlists.json file, which defines every Type, Class, Letter Group, and Lesson entry in the MPP user interface. The file is organized by class, starting with BC1 and ending with OverLearn.

A typical entry looks like this:

```
{"sort": 1, "userTarget": "STUDENT", "class": "BC1", "letterGroup":
"REA",
 "newlineChunking": false, "display": "REA", "fileName":
"SB1REA1.json"}
```

Field descriptions:

- "sort" — Not currently in use.
- "userTarget" — Determines whether the entry appears under the Student or Instructor TYPE in the MPP.
- "class" — Controls what appears in the CLASS dropdown. Adding a new value here creates a new class in the MPP interface.
- "letterGroup" — Sets what is displayed in the LETTER GROUP dropdown. Typically used to group related content files.
- "newlineChunking" — When true, it turns KEEP LINES on. When false, it turns it off.
- "display" — The label shown in the LESSON dropdown. Should reflect the content (e.g., "2 letter words," "4 word sentences").
- "fileName" — The name of the associated content file. This value is case-sensitive.

SECTION 2: CONTENT DEVELOPMENT

General Guidelines

- All content files are stored in the wordfiles folder.

- The current naming convention is to begin the filename with the class level, followed by a short identifier for the exercise.
- Every line of text must end with a space. Without the trailing space, the line will run together with the next line during playback.
- The VOICE and SPELL settings in the MPP control whether code is spoken and whether it is spelled out character by character or spoken as a word(s).

Simple Exercises

Simple exercises use either .json files or plain text files with no special internal coding.

.json Format

.json files are used to generate random character groups and define how long an exercise runs. They are primarily used in BC1 and BC2.

Example — B1.json:

```
{
  "letters":        "reatinpgslcdhofuwb",
  "minWordSize":     3,
  "maxWordSize":     3,
  "practiceSeconds": 120
}
```

- "letters" — All characters to be included in the exercise.
- "minWordSize" — Minimum number of characters per group.
- "maxWordSize" — Maximum number of characters per group.
- "practiceSeconds" — Total exercise duration in seconds.

Plain Text Format

Plain text files are used when no voice or grouping overrides are needed. Content can be arranged several ways:

Characters on a single line (sent individually):

```
H O F O F H
```

Characters on separate lines (sent individually):

```
H
O
F
```

Words on a single line (words sent individually):

```
RUB BAR RAW FOR WAR ARE
```

Phrases on separate lines (words sent as a phrase):

```
I THINK SO
LOOK AT THIS
GIVE ME THAT
```

Exercises with Voice Overrides

The wordify.json file defines how characters and abbreviations are spoken by default. In some cases, a content file must override that default. For example, "on" defaults to "Ontario." In a sentence context, you would want it spoken as "on."

The override syntax uses curly braces with a pipe separator:

```
{on| on}
```

Another use case: if you want the VOICE to speak characters individually rather than as a word:

```
{one| o n e}   ← spoken as three separate characters (note spaces
between them)
```

In all cases above, the code is sent first, then spoken. You can also suppress voice output entirely for a specific sequence:

```
{one| }   ← sends the code but speaks nothing
```

> *This silent voice technique is particularly useful in complex exercise coding, as shown in the next sections.*

Random Groups (Shuffle)

To keep sets of words, phrases, or sentences grouped together while allowing those groups to appear in random order each run, enable shuffling. This can be done in the preset file or manually via the Shuffle toggle in the MPP:

```
{
  "key":     "isShuffledSet",
  "value":   true,
  "comment": null
},
```

In the content file, add a second pipe and a group number. All entries sharing the same number will be kept together and presented as a unit. A space must be added between curly braces:

```
{JJJ |JJJ |10} {JAM |JAM |10} {BBB |BBB |2} {BET |BET |2}
```

To shuffle the order of items within a group (intra-group shuffle), set shuffleIntraGroup to true. This option is currently hidden in the standard MPP MAIN and DEV interface to prevent unintentional use, but can be accessed for testing at:

```
https://longislandcw.github.io/morsebrowser/dev/?adminMode=1
```

It appears next to REPEAT SPACING. The content coding is the same as above. The default setting is “false”.

```
{
  "key":     "shuffleIntraGroup",
  "value":   false,
  "comment": null
}
```

Combining Voice and Random Groups

Voice overrides and random grouping can be combined to create a send code-then speak -then repeat code pattern. This is an exception to the usual rule of keeping groups on separate lines.

In the example below, each phrase is structured in three parts that share the same group number:

- Part 1: Send the code and speak the phrase.
- Part 2: Send the code (no voice).
- Part 3: Send VV as a separator (no voice).

```
{GIVE YOUR BEST|GIVE YOUR BEST|274} {GIVE YOUR BEST||274}
{VV||274}
{LOW HANGING FRUIT|LOW HANGING FRUIT|449} {LOW HANGING
FRUIT||449} {VV||449}
```

Making Complex Content Files with Excel

For exercises with many entries, it is more practical to generate the coded content in some automated way. I create an Excel spreadsheet and then paste the results into Word to make a text file. The following examples show the Excel formula approach.

Example 1 — Basic Random Groups

Excel setup (columns A = group number, B = content, C = formula):

A	B	C
1	AN	="{" & B1 & "│" & B1 & "│" & A1 & "} "
1	AT	="{" & B2 & "│" & B2 & "│" & A2 & "} "
2	BE	="{" & B3 & "│" & B3 & "│" & A3 & "} "
2	BY	="{" & B4 & "│" & B4 & "│" & A4 & "} "

Resulting output:

```
{AN │AN │1}
{AT │AT │1}
{BE │BE │2}
```

```
{BY │BY │2}
```

Example 2 — Voice + Random Groups Combined

Excel setup for the send-then speak-then repeat pattern:

A	B	C
1	I THINK SO	="{" & B1 & "│" & B1 & "│" & A1 & "} " & "{" & B1 & "││" & A1 & "} " & "{VV││" & A1 & "} "
2	LOOK AT THIS	="{" & B2 & "│" & B2 & "│" & A2 & "} " & "{" & B2 & "││" & A2 & "} " & "{VV││" & A2 & "} "
3	GIVE ME THAT	="{" & B3 & "│" & B3 & "│" & A3 & "} " & "{" & B3 & "││" & A3 & "} " & "{VV││" & A3 & "} "
4	I AM HERE	="{" & B4 & "│" & B4 & "│" & A4 & "} " & "{" & B4 & "││" & A4 & "} " & "{VV││" & A4 & "} "

Resulting output (all on one line in the text file):

```
{I THINK SO │I THINK SO │9} {I THINK SO ││9} {VV││9} {LOOK AT THIS │LOOK AT THIS │12} {LOOK AT THIS ││12} {VV││12} {GIVE ME THAT │GIVE ME THAT │7} {GIVE ME THAT ││7} {VV││7} {I AM HERE │I AM HERE │9} {I AM HERE ││9} {VV││9}
```

Page 1: [1] Deleted	**Mike Padron**	**4/12/26 11:11:00 AM**

Page 2: [2] Deleted	**Mike Padron**	**4/12/26 10:11:00 AM**